

Adaptive ETA Prediction for Inland Waterway Cargo Logistics Using Machine Learning, NLP, and Reinforcement Learning

Riyan Wankhede (23BCE9287)

Department of Computer Science and Engineering
Vellore Institute of Technology, Andhra Pradesh
(VIT-AP)
riyanwankhede@gmail.com

Abstract – Accurate Estimated Time of Arrival (ETA) prediction is a critical operational challenge in inland waterway cargo logistics, where dynamic environmental and operational factors cause substantial schedule deviations. This paper presents an ETA prediction framework that combines a Deep Neural Network (DNN) for base ETA regression with a three-tier Natural Language Processing (NLP) pipeline that extracts a delay-severity score from unstructured operational logs, and compares two ways of using that score to correct the base prediction: a tabular Q-learning agent and a two-parameter linear correction. On a synthetic dataset of 1,000 voyages, evaluated with a train/validation/test split over five random seeds, the DNN reaches a mean test MAE of 152.9 min. The linear severity correction reduces MAE by 7.6% on average (to 141.5 min), whereas the Q-learning agent does not improve on the DNN (−0.4%). Text-derived delay signals therefore carry useful information, and because each ETA adjustment is a single one-step decision, a direct regression on the signal uses it more effectively than tabular reinforcement learning.

Keywords – ETA prediction, inland waterway logistics, deep learning, NLP, Q-learning, reinforcement learning, DistilBERT, contextual bandits.

I. INTRODUCTION

Inland waterway transport (IWT) represents a sustainable, high-capacity cargo mode that is increasingly important for continental logistics networks. Unlike road or rail freight, IWT is acutely sensitive to environmental variability—river currents, weather conditions, and lock-gate congestion can produce arrival-time deviations of several hours. Port terminals, fleet operators, and cargo planners therefore depend on accurate and up-to-date ETA information to minimize dwell costs, optimize berth allocation, and maintain supply-chain reliability.

Traditional ETA prediction systems rely on deterministic routing formulae or shallow regression models trained on structured voyage records. These approaches share two fundamental limitations. First, they cannot incorporate the rich contextual information contained in unstructured operational text—captain’s logs, port authority notices, and maintenance bulletins that are routinely produced but seldom exploited. Second, they generate static predictions that cannot self-correct as voyage conditions evolve.

This paper addresses both limitations with a three-component system. A DNN trained on five structured voyage features provides a base ETA. An NLP pipeline extracts a numerical delay-severity score from free-text operational reports. A correction stage then adjusts the base ETA using that score; we compare a Q-learning agent with a two-parameter linear correction and a constant-bias control. The system is evaluated on a 1,000-voyage synthetic dataset with a held-out test set and five random seeds.

II. RELATED WORK

ETA prediction for maritime and inland waterway transport has received growing attention, and a recent review surveys methods for waterway networks [1]. Noman et al. proposed a data-driven GRU model for vessel ETA on inland natural and

artificial waterways [2]. Park et al. combined AIS data with a path-finding algorithm to predict vessel ETA [3], and Kolley et al. used machine-learned arrival-time predictions for robust berth scheduling [4].

Deep learning has also been applied to ship transit-time prediction [5]. Our base predictor is a feed-forward network with batch normalisation [6] and dropout [7], trained with the Adam optimiser [8] in PyTorch [9].

Transformer language models such as BERT [10] and its distilled variant DistilBERT [11] are widely used for short-text classification. In the maritime domain, text mining of accident reports has been used to identify the causes of ship collisions [12]. Pipelines that try inexpensive rules before a neural model trade coverage against computational cost; our NLP module follows this pattern.

Q-learning [13] and its deep extension, the Deep Q-Network (DQN) [14], are standard methods for sequential decision problems [15], and DQN has been applied to berth allocation in container terminals [16]. When each decision is made once, with no subsequent state, the problem is a contextual bandit [17]; we return to this distinction in Section VI.

III. SYSTEM ARCHITECTURE

The proposed system comprises three loosely coupled modules that operate in sequence for each voyage prediction request (Fig. 1). The DNN module ingests five structured features and outputs a base ETA in minutes. The NLP module processes any available free-text operational report and returns a scalar delay-severity score $s \in [0, 1]$. The correction module receives both outputs and adjusts the base ETA, either with a Q-learning agent that selects a proportional adjustment or with a linear function of the severity score.

DNN Module. The regression DNN is implemented in PyTorch and trained with mean-squared-error loss. It serves as the primary ETA estimator and is frozen while the correction stage is fitted.

NLP Module. A three-tier pipeline processes maritime text. Tier 1 applies phrase-level shortcut rules for high-confidence patterns (e.g., "no issues" $\rightarrow$ 0.05). Tier 2 computes a weighted average over a 33-keyword maritime lexicon with empirically determined severity weights. Tier 3 invokes a DistilBERT-based sentiment model as a fallback when no phrase or keyword matches.

RL Module. A tabular Q-learning agent maintains a Q-table over 210 states and 5 actions. At inference time, the agent selects the greedy action (highest Q-value) to produce the adjusted ETA. The linear correction, fitted by least squares on training voyages, is evaluated alongside it (Section IV-E).



Fig. 1. End-to-End Pipeline

IV. METHODOLOGY

A. Dataset

A synthetic dataset of 1,000 voyage records was generated to represent inland waterway operating conditions. Each record contains five input features (Table I) and a ground-truth ETA in minutes. Each voyage also carries a hidden disruption level in $[0, 1]$ that adds up to 30% to its travel time and is never given to the DNN; it selects the voyage's operational log from a pool of 20 messages, whose NLP severity is the only signal the correction stage sees. The dataset was partitioned into 640 training, 160 validation and 200 test voyages. Features were standardised using training-set statistics (zero mean, unit variance) before being passed to the DNN.

Feature	Range
Distance (km)	50 – 500
Vessel speed (km/h)	5 – 20
River current (m/s)	0 – 2
Weather severity	0 – 1
Port congestion	0 – 1

TABLE I. Dataset Feature Statistics

B. Phase 1 - Deep Neural Network ETA Predictor

The DNN is implemented in PyTorch [9] as a three-layer MLP. The architecture is:

Input (5) $\rightarrow$ Dense-128 (ReLU) $\rightarrow$ Dense-64 (ReLU) $\rightarrow$ Dense-32 (ReLU) $\rightarrow$ Output (1)

The model has 11,585 trainable parameters. Each hidden layer uses batch normalisation [6] and dropout [7] (rates 0.30, 0.30 and 0.15). Training used the Adam optimiser [8] with learning rate 1×10^{-3} . Training ran for up to 150 epochs with batch size 32 and early stopping (patience = 20 epochs on the validation split).



Fig. 2. Phase 1- Deep Neural Network

C. Phase 2 - Cascaded NLP Severity Extractor

The NLP pipeline processes free-text maritime logs through a three-tier cascade to balance speed and accuracy:

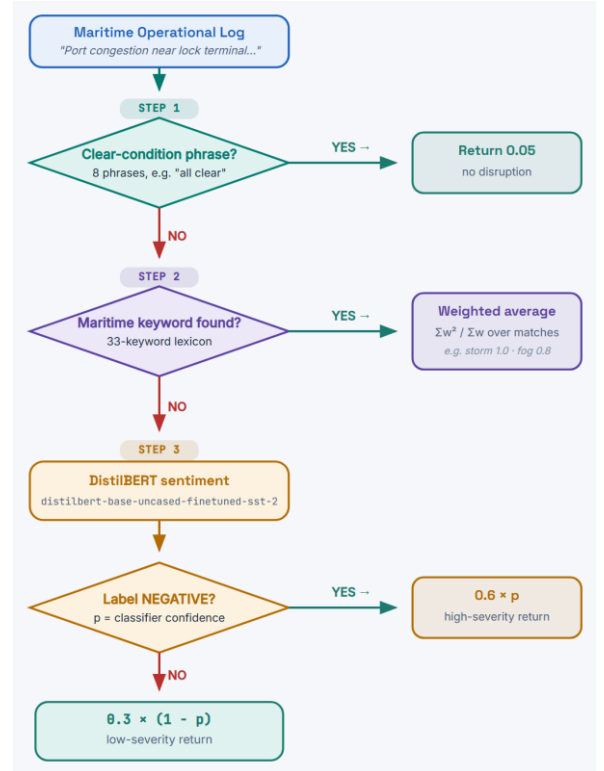


Fig. 3. Phase 2 - NLP Severity Extractor

Tier 1 - Clear-phrase shortcut.

Eight hand-crafted phrases indicating zero-delay conditions (e.g., 'smooth navigation', 'no issues') are matched by exact substring search. A match immediately returns severity = 0.05, bypassing downstream models.

Tier 2 - Keyword weighted average.

A domain lexicon of 33 maritime terms is defined with empirically calibrated severity weights (e.g., *storm* $\rightarrow$ 1.0, *congestion* $\rightarrow$ 0.9, *fog* $\rightarrow$ 0.8, *maintenance* $\rightarrow$ 0.75). All matching keywords are identified via substring search, and their weights are combined in a weight-weighted average ($\Sigma w^2 / \Sigma w$), which is returned as the severity score.

Tier 3 - DistilBERT fallback.

When no phrase or keyword matches, DistilBERT [11], fine-tuned on SST-2, performs sentiment classification. A NEGATIVE label with probability p maps to severity $0.6p \in [0.3, 0.6]$; a POSITIVE label maps to $0.3(1 - p)$, in $[0, 0.15]$.

Log Entry (excerpt)	Severity
Smooth navigation, no operational issues	0.05
Heavy fog reducing visibility	0.80
Port congestion, lock maintenance	0.86
Storm conditions delaying movement	0.88
Strong opposing river current	0.65

TABLE II. NLP Severity Scores for Representative Logs

D. Phase 3 - Q-Learning Adaptive Adjustment

The RL agent is a tabular Q-learning agent [13] over a discrete state-action space.

The state s is defined as a tuple (η, σ) , where η is the base ETA discretised into 10 equal-width buckets over the 1st–99th percentile range of training-voyage ETAs and σ is the severity score discretised into 21 bins. This yields $|S| = 10 \times 21 = 210$ states.

$$A = \{0\%, +2\%, +5\%, +10\%, +20\%\} \times \text{eta}_{\text{base}}$$

The five actions represent ETA adjustments as percentages of the base ETA, ensuring scale-invariant adjustments. The reward function incentivises accuracy:

$$r = \frac{(|\text{err}_{\text{base}}| - |\text{err}_{\text{adj}}|)}{\text{eta}_{\text{base}}}$$

Where,

$$\text{err}_{\text{base}} = \text{eta}_{\text{base}} - \text{eta}_{\text{actual}}, \text{err}_{\text{adj}} = \text{eta}_{\text{adj}} - \text{eta}_{\text{actual}}$$

Positive rewards are earned when the adjustment reduces error; negative rewards penalise overcorrection.



Fig. 4. Q-Table Heatmap and Action Space

Q-values are updated via the standard Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Each episode consists of a single voyage and a single action, after which it terminates, so the bootstrap term $\gamma \max Q(s', a')$ is zero and the update reduces to $Q(s, a) \leftarrow Q(s, a) + \alpha [r - Q(s, a)]$. The problem is therefore a contextual bandit [17] rather than a sequential decision process.

Hyperparameters were set as: $\alpha = 0.10$ (learning rate), $\gamma = 0.90$ (discount factor; unused, since each episode has one step), ϵ -greedy exploration from $\epsilon = 1.0$ decaying by 0.003 per episode to a floor of 0.05. The Q-table was initialised with small positive values (0.01) to encourage early exploration. Training ran for 5,000 episodes, sampled from training voyages only.

Episodes	5,000	ϵ start + min	1.00 - 0.05
State space	$10 \times 21 = 210$	ϵ decay rate	0.003 exponential
Actions	5 proportional fractions	Q initialise	0.01 (optimistic)
α (learn rate)	0.10	States visited	79 / 210 (37.6%)
γ (discount)	unused: 1-step episodes	Q-max achieved	0.0675

Fig. 5. Hyperparameter Summary

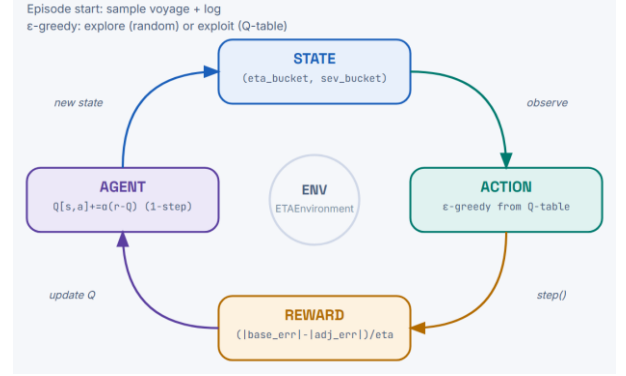


Fig. 6. Phase 3 - Q-Learning Reinforcement Learning Agent

E. Linear Severity Correction and Baselines

As a direct alternative to the agent, the base ETA is corrected as $\text{ETA}_{\text{adj}} = \text{ETA}_{\text{base}} \times (1 + a + b \cdot s)$, where s is the NLP severity score and a, b are fitted by least squares to the training voyages' relative error $(\text{ETA}_{\text{actual}} / \text{ETA}_{\text{base}} - 1)$. As a control that uses no text, a constant factor (the median relative error on training voyages) is also applied. Averaged over seeds, the fitted coefficients are $a = -0.085$ and $b = +0.120$.

F. Evaluation Protocol

The 1,000 voyages are split into 640 training, 160 validation and 200 test voyages. The DNN is selected by early stopping on the validation split; the Q-learning agent and both corrections are fitted on training voyages only; and each test voyage's severity comes from its own log text. The whole pipeline is repeated with five random seeds (42, 1, 2, 3, 4), and each method's gain is computed against the DNN from the same seed, because the absolute MAE varies by about ± 16 min between seeds.

V. EXPERIMENTAL RESULTS

A. Phase 1 DNN Performance

With seed 42, the DNN stops early at epoch 59 and reaches a test MAE of 153.61 min and R^2 of 0.9686. Over five seeds, its test MAE is 152.9 ± 16.4 min.

B. Phase 2 NLP Pipeline Behaviour

Of the 20 log messages used in the experiments, Tier 1 handles 3 (15%) and Tier 2 handles 17 (85%); none reaches Tier 3. Two additional demonstration messages without phrases or keywords exercise the Tier-3 path (severities 0.60 and 0.00 for a negative and a positive message). On a laptop CPU, Tiers 1–2 take about 13 μ s per message and the DistilBERT tier about 10.5 ms. The severity scores have not been validated against human annotations; Table II lists representative scores.

C. RL Agent Training Convergence

With seed 42, the rolling mean reward rose from -0.013 over the first 1,000 episodes to about -0.002 by episode 5,000, but did not become positive. The agent visited 79 of 210 states (37.6%), with a maximum Q-value of 0.0675; the visited states concentrate in the few severity buckets produced by the 20 log messages (Fig. 4).

D. Comparison of Correction Methods

Table III summarises the five-seed results. The linear severity correction gives the largest and most consistent reduction in MAE (7.6% on average) and beats Q-learning in 4 of 5 seeds; Q-learning does not improve on the DNN on average, and the constant correction recovers only a small part of the gain (Fig. 7). With seed 42, most of the linear correction's gain comes from low-disruption voyages, on which the DNN over-predicts (MAE 170 $\rightarrow$ 87 min).

Method	MAE (min)	Gain vs DNN
DNN alone	152.9 $\pm$ 16.4	—
+ constant bias	151.5 $\pm$ 17.7	+1.0% $\pm$ 3.5%
+ NLP $\rightarrow$ Q-learning	153.7 $\pm$ 19.0	-0.4% $\pm$ 2.8%
+ NLP $\rightarrow$ linear	141.5 $\pm$ 19.6	+7.6% $\pm$ 5.4%

TABLE III. Test MAE over Five Seeds (Mean $\pm$ SD)

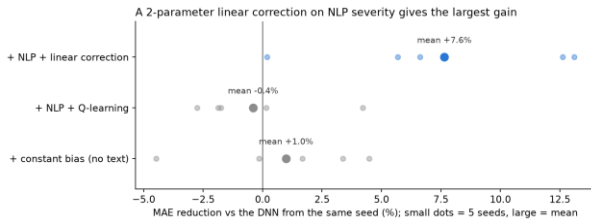


Fig. 7. MAE Reduction vs the DNN over Five Seeds

VI. DISCUSSION

The results support a narrower claim than the original hypothesis. The NLP-derived severity signal is useful: a two-parameter linear correction on it lowers ETA error by 7.6% on average, about 11 minutes per voyage. Q-learning on the same signal does not improve on the DNN. The adjustment is made once per voyage and nothing follows it, so the task is a contextual bandit [17]; a tabular agent must learn separate values for 210 states and 5 coarse actions from 5,000 noisy one-step rewards, whereas the linear correction fits two parameters to all training voyages at once.

The three-tier NLP pipeline is fast: the rule-based tiers handle all 20 experimental messages in microseconds, and the DistilBERT fallback is reserved for messages without known phrases or keywords.

Several limitations apply. First, the dataset is synthetic, and each voyage's log is drawn from a pool of 20 messages matched to its hidden disruption level, so the text signal is cleaner than real operational text; validation on historical voyage records is required. Second, the NLP severity scores have not been validated against human annotations. Third, the spread across seeds is large relative to the effects (± 5.4 points for the linear gain), so larger test sets are needed for precise estimates. Fourth, the agent's action space only allows upward adjustments.

Future work will:

(i) collect real operational logs from an IWT operator for end-to-end validation.

- (ii) annotate a set of real logs to validate the severity scores.
- (iii) evaluate contextual-bandit algorithms and regression on full text embeddings in place of tabular Q-learning.
- (iv) explore fine-tuning a domain-specific maritime language model to improve NLP tier-3 accuracy.

VII. CONCLUSIONS

This paper presented an ETA prediction framework for inland waterway cargo logistics that combines a DNN base predictor with an NLP delay-severity signal. Evaluated with a held-out test set over five seeds, a two-parameter linear correction on the severity score reduces MAE from 152.9 to 141.5 min (-7.6%), while a tabular Q-learning agent using the same signal gives no reliable gain. Text-derived delay signals therefore carry useful information for ETA prediction, and one-step adjustment problems are better served by direct regression than by tabular reinforcement learning.

The framework is modular: each component can be upgraded independently, enabling a clear path toward real-world deployment and further performance gains through domain-specific data collection and richer correction models.

VIII. REFERENCES

- [1] A. A. Noman, A. Heuermann, S. Wiesner, and K.-D. Thoben, "A review of vessel time of arrival prediction on waterway networks: Current trends, open issues, and future directions," *Computers*, vol. 14, no. 2, Art. no. 41, 2025, doi: 10.3390/computers14020041.
- [2] A. A. Noman, A. Heuermann, S. A. Wiesner, and K.-D. Thoben, "Towards data-driven GRU based ETA prediction approach for vessels on both inland natural and artificial waterways," in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, 2021, pp. 2286–2291, doi: 10.1109/ITSC48978.2021.9564883.
- [3] K. Park, S. Sim, and H. Bae, "Vessel estimated time of arrival prediction system based on a path-finding algorithm," *Maritime Transport Research*, vol. 2, Art. no. 100012, 2021, doi: 10.1016/j.martra.2021.100012.
- [4] L. Kolley, N. Rückert, M. Kastner, C. Jahn, and K. Fischer, "Robust berth scheduling using machine learning for vessel arrival time prediction," *Flexible Services and Manufacturing Journal*, vol. 35, no. 1, pp. 29–69, 2023, doi: 10.1007/s10696-022-09462-x.
- [5] S.-L. Yoo and K.-I. Kim, "Deep learning-based prediction of ship transit time," *Ocean Eng.*, vol. 280, Art. no. 114592, 2023, doi: 10.1016/j.oceaneng.2023.114592.
- [6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1412.6980.
- [9] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8024–8035.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language

- understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.
- [11] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.
- [12] J. Yu, Z. Wu, W. Liu, and W. Zhao, "Identifying the causes of ship collisions accident using text mining and Bayesian networks," *Elektronika ir Elektrotechnika*, vol. 29, no. 6, pp. 58–67, 2023, doi: 10.5755/j02.eie.35630.
- [13] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3–4, pp. 279–292, 1992, doi: 10.1007/BF00992698.
- [14] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [15] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [16] P. Wang, J. Li, and X. Cao, "Discrete dynamic berth allocation optimization in container terminal based on deep Q-network," *Mathematics*, vol. 12, no. 23, Art. no. 3742, 2024, doi: 10.3390/math12233742.
- [17] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 661–670, doi: 10.1145/1772690.1772758.